Assignment 1 Advanced Software Development for Robotics - Group 24

Luuk Dijkman, s27262129
Jesse Oudijk, s2809206

March 1, 2025

Assignment 1.1: Image Processing with ROS2

Camera input with standard ROS2 tools

Both `depth` and `history` are quality of service policies for subscribers and publishers. Messages you want to publish or subscribe to are stored in a queue. The `history` parameter has as default value, *keep last*, which means the last messages that you want to publish or subscribe to are stored in a queue. The other option is *keep all*, which means that all messages are stored in the queue before they are being published or handled by the subscriber. The `depth` parameter is only considered when `history` is set to *keep last* and it determines the queue size of the publisher and subscriber, its default value is 10. When you want to publish messages faster than ROS can handle, ROS will start dropping the oldest messages based on the `depth` value. For a subscriber, if you are receiving messages faster than you can handle and new messages are being received, ROS will start dropping the oldest messages again. Generally, a sequence controller has to deal with changing desired states, therefore setting `history` to *keep last* is desired because then it is possible for the system to start following a new sequence of setpoints to a new desired state. Setting `depth` to 1 means the publisher will publish the most recently computed message and the subscriber will handle the most recently published message. This allows for quick response times in case of emergency and near collision.

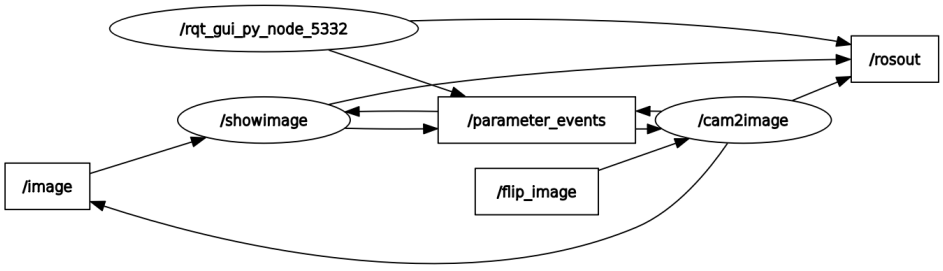

Figure 1: Rqt graph with the cam2image and showimage nodes running.

Figure 1 shows the rqt graph, where the `cam2image` and `showimage` are running. It shows that `cam2image` publishes the image messages to the `image` topic and `showimage` is subscribed to this topic.

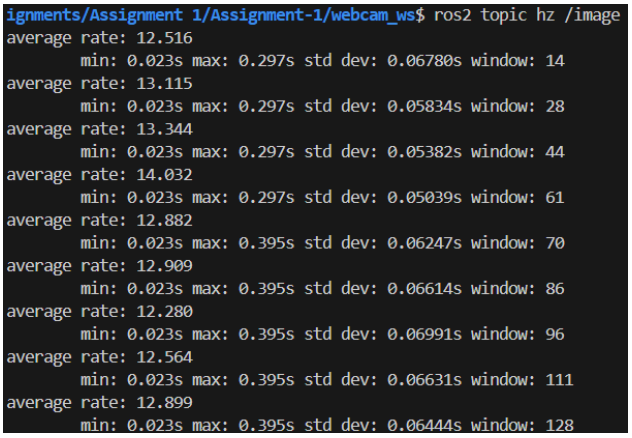

Figure 2: Framerate of published image messages on /image topic.

Figure 2 shows the framerate of the generated messages on the `image` topic. The framerate of the generated messages is influenced by the frequency at which the messages are published. You can influence the framerate by ensuring that the ROS2 traffic is not too crowded and changing the frequency at which the publisher is called. The maximum framerate is the frequency at which the publisher is called, which is set to $30Hz$ in this case.

Adding a brightness node

To compute the average brightness of the image, the node `brightness`, was created. This node converts the ROS2 image message into an `OpenCV` [1] image. It then loops through all pixels, determines their brightness by adding the R, G and B values and dividing by three. Then the average brightness of the whole image is calculated and logged to the terminal. The brightness is then compared to a threshold `brightness_threshold` and logs whether the light is on, based on whether the brightness is larger than the threshold or not.

```
[INFO] [1740746876.144499271] [brightness]: Brightness: 239.125900
[INFO] [1740746876.144669984] [brightness]: Light is on
[INFO] [1740746876.183326966] [brightness]: Brightness: 228.186447
[INFO] [1740746876.183518188] [brightness]: Light is on
[INFO] [1740746876.215143589] [brightness]: Brightness: 131.643814
[INFO] [1740746876.215318974] [brightness]: Light is on
[INFO] [1740746876.243884941] [brightness]: Brightness: 131.643814
[INFO] [1740746876.244085225] [brightness]: Light is on
[INFO] [1740746876.279965082] [brightness]: Brightness: 94.426476
[INFO] [1740746876.280124978] [brightness]: Light is off
[INFO] [1740746876.314177483] [brightness]: Brightness: 69.988121
[INFO] [1740746876.314426870] [brightness]: Light is off
[INFO] [1740746876.341926776] [brightness]: Brightness: 69.988121
[INFO] [1740746876.342083036] [brightness]: Light is off
[INFO] [1740746876.377749413] [brightness]: Brightness: 38.817230
[INFO] [1740746876.377926266] [brightness]: Light is off
[INFO] [1740746876.412227815] [brightness]: Brightness: 15.865178
[INFO] [1740746876.412393056] [brightness]: Light is off
[INFO] [1740746876.444523952] [brightness]: Brightness: 23.879162
[INFO] [1740746876.444695637] [brightness]: Light is off
[INFO] [1740746876.475282309] [brightness]: Brightness: 23.879162
[INFO] [1740746876.475460181] [brightness]: Light is off
[INFO] [1740746876.512286403] [brightness]: Brightness: 30.804457
[INFO] [1740746876.512418298] [brightness]: Light is off
[INFO] [1740746876.531114195] [brightness]: Brightness: 121.329224
[INFO] [1740746876.531297769] [brightness]: Light is on
[INFO] [1740746876.574503732] [brightness]: Brightness: 219.194290
[INFO] [1740746876.574610264] [brightness]: Light is on
[INFO] [1740746876.607580755] [brightness]: Brightness: 240.627670
[INFO] [1740746876.607753788] [brightness]: Light is on
[INFO] [1740746876.641398251] [brightness]: Brightness: 244.586838
[INFO] [1740746876.641557632] [brightness]: Light is on
```

Figure 3: Logging output in terminal when running brightness node and cam2image.

Figure 3 shows the output of the brightness node that analyses `cam2image` images. It is tested by placing a hand in front of the webcam while using a brightness threshold default value of 100. It can be seen that when the hand is placed on the webcam, the brightness decreases and the node detects that the brightness falls below the threshold and outputs that the light is off. When the brightness increases to above 100, the message shows that indeed the light has turned back on.

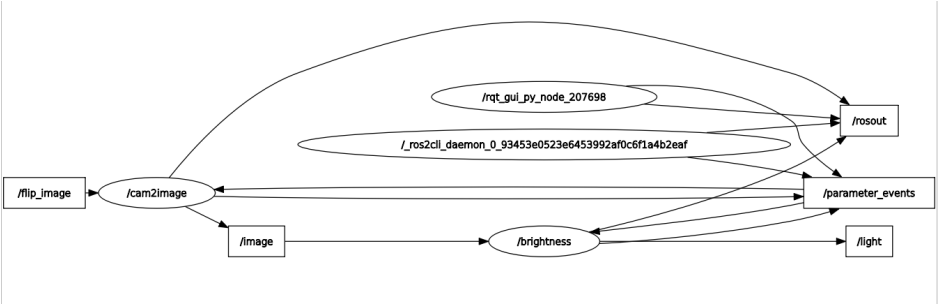

Figure 4: Rqt graph of the brightness node and cam2image node running.

Figure 4 shows the rqt graph with the `/brightness` and `/cam2image` node running. The `/cam2image` node provides the `/brightness` node with the images through the `/image` topic. The `/brightness` node publishes the status of the light to the `/light` topic. Both nodes are continuously listening to the `/parameter_events` topic for potential parameter changes.

Adding ROS2 parameters

To ensure that `brightness_threshold` is a ROS2 parameter, it is initialized in the header file and also declared in the `parse_parameters()` function with a default value of 100 using the following line of code:

```
brightness_threshold = this->declare_parameter("brightness_threshold", 100);
```

This makes it already possible to set the parameter when starting the node, but to make sure that is settable during run time, the node must be able to see changes in the parameter. This is done using the following line, which is ran in the `avg_brightness()` function:

```
brightness_threshold = this->get_parameter("brightness_threshold").as_int();
```

```
ignments/Assignment 1/Assignment-1/webcam_ws$ ros2 run brightness_pkg brightness --ros-args -p brightness_threshold:=200
[INFO] [1740748394.242618274] [brightness]: Brightness: 198.221420
[INFO] [1740748394.242897920] [brightness]: Light is off
[INFO] [1740748394.246190334] [brightness]: Brightness: 198.170944
[INFO] [1740748394.246307941] [brightness]: Light is off
[INFO] [1740748394.269950112] [brightness]: Brightness: 198.014267
[INFO] [1740748394.270121308] [brightness]: Light is off
[INFO] [1740748394.296114634] [brightness]: Brightness: 197.939682
[INFO] [1740748394.296248869] [brightness]: Light is off
[INFO] [1740748394.325531156] [brightness]: Brightness: 197.939682
[INFO] [1740748394.325714661] [brightness]: Light is off
[INFO] [1740748394.359733841] [brightness]: Brightness: 197.847855
```

Figure 5: Logging output in terminal when running brightness node and cam2image and the brightness threshold is set when the node is started.

```
[INFO] [1740748397.388670507] [brightness]: Brightness: 6.127487
[INFO] [1740748397.388805688] [brightness]: Light is off
[INFO] [1740748397.420219547] [brightness]: Brightness: 40.025921
[INFO] [1740748397.420349282] [brightness]: Light is off
[INFO] [1740748397.461376220] [brightness]: Brightness: 40.025921
[INFO] [1740748397.461544928] [brightness]: Light is off
[INFO] [1740748397.513002060] [brightness]: Brightness: 131.119797
[INFO] [1740748397.513184105] [brightness]: Light is off
[INFO] [1740748397.571314029] [brightness]: Brightness: 131.119797
[INFO] [1740748397.571494496] [brightness]: Light is off
[INFO] [1740748397.595692152] [brightness]: Brightness: 246.283569
[INFO] [1740748397.595880357] [brightness]: Light is on
[INFO] [1740748397.600062698] [brightness]: Brightness: 248.892883
[INFO] [1740748397.600158602] [brightness]: Light is on
[INFO] [1740748397.644413989] [brightness]: Brightness: 248.892883
[INFO] [1740748397.644575840] [brightness]: Light is on
[INFO] [1740748397.677411951] [brightness]: Brightness: 248.903580
[INFO] [1740748397.677593995] [brightness]: Light is on
[INFO] [1740748397.711160627] [brightness]: Brightness: 249.634491
[INFO] [1740748397.711336235] [brightness]: Light is on
```

Figure 6: Logging output in terminal when running brightness node and cam2image and the brightness threshold is set when the node is started.

Figure 5 shows that when the threshold is set to 200, the light is indeed turned off for values below the threshold right away. In Figure 6, it can be seen that for values above the threshold, the node still detects that the light is on.

```
ignments/Assignment 1/Assignment-1/webcam_ws$ ros2 param set \brightness brightness_threshold 200
Set parameter successful
```

Figure 7: Command used to change set brightness threshold parameters while the node is running

Figure 8: Logging output in terminal when running brightness node and cam2image and the brightness threshold is set while the node is running.

Figure 7 shows that the parameter is set successfully while running the node normally. In Figure 8, the result of the change in brightness threshold can be seen. While the brightness keeps approximately the same value, the threshold has changed midway and the light has changed from on to off. The rqt graph of this assignment with the added ROS2 parameter is the same as for the previous assignment, which is seen in Figure 4.

Simple object-position indicator

To indicate the position of an object on a webcam image, the `OpenCV` library has been used. This as it provides functionality to gray scale an image [2], threshold it [3], and determine the image moment [4]. From this image moment the center of gravity can be determined, as described in [5]. Finally, `OpenCV` provides the `cv::imshow` function [6] to be able to visualise the final results. An example of such an image can be seen in Figure 9.

For the threshold a ROS parameter was used, to allow for easy user adaptability. If the environment or webcam is changed the user can opt to change its value to allow for a balance between accuracy and sensitivity. To ensure always the newest parameter value is used the following was stated before the threshold is applied:

```
gray_threshold_ = this->get_parameter("gray_threshold").as_int();
cv::Mat thresholded_image;
cv::threshold(gray_image, thresholded_image, gray_threshold_, 255, cv::THRESH_BINARY);
```

A binary threshold was used as can be seen by the code snippet. This was done to create a binary image, including white and black dots for pixels that did or did not exceed the threshold respectively.

After some trial and error by changing the threshold, a value of 250 was found for the given webcam and environment. The result can be seen in Figure 9.

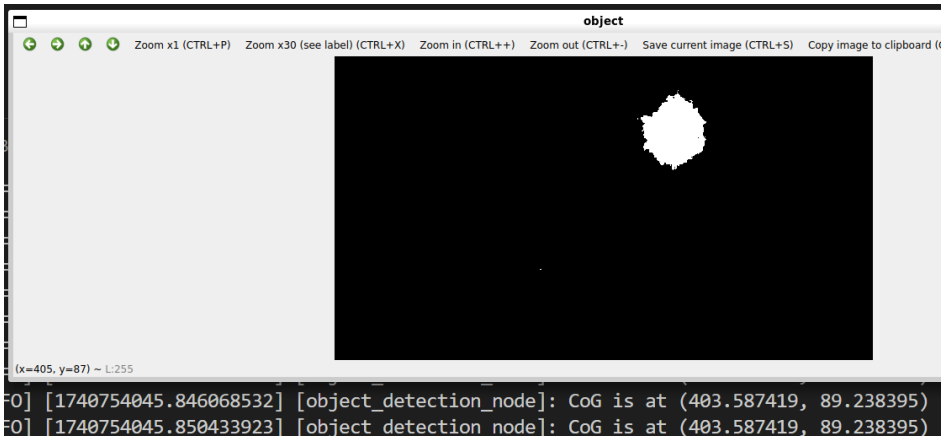

Figure 9: Screenshot of the thresholded image created by shining at the webcam with a torch.

At the bottom of Figure 9, the center of gravity published by the `object detection node` can be seen. In the bottom left of the image environment also the (x, y) pixel location of the mouse cursor can be seen. The cursor was placed at approximately the center of the white blob seen in Figure 9 (unfortunately a screenshot does not show where the cursor is located). As the CoG published is close to the placement of the cursor, it is believed that the code functions appropriately.

However, it must be noted that from testing in different lighting environments, the current approach is quite sensitive to outside light sources. Even very high treshold values are sometimes not able to eliminate all the noise inside brightly lit environments. It is hence questionable whether it would be suitable for the tracking of a green ball. Potential future improvements could include automatic threshold calibration, and getting rid of the OpenCV functionality as it is yet uncertain whether it would be able to run accordingly on the RELbot.

The rqt graph of the object detection node and the cam2image node running can be seen in Figure 10.

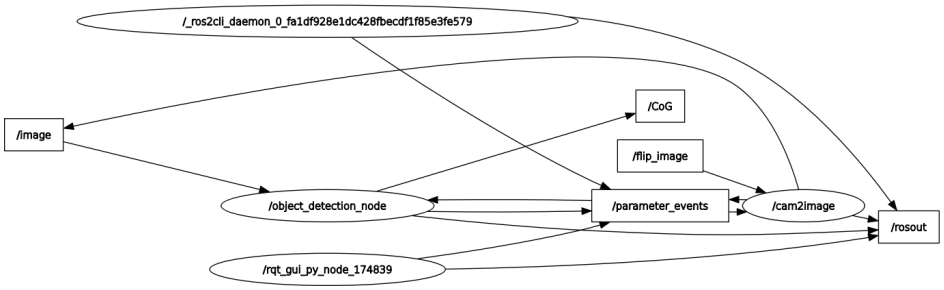

Figure 10: Rqt graph of the object detection node and the cam2image node running.

Figure 10 shows that object detection node is subscribed to the `\image` topic to receive images from the `cam2image` node. It also shows that the objection detection node publishes to the center of gravity topic.

Assignment 1.2: Sequence controller

Unit test the sequence controller using the RELbot Simulator

The sequence generator node creates a twist message publisher to the `input/twist` topic that the `relbot_simulator` is subscribed to. It then uses a timer that runs every $0.5s$ to call the publisher function. In this function, the sequence is determined. First, a linear velocity in x-direction of 1 m/s is published for 5 s, this direction

introduces the zooming of the subimage. Then, the same velocity is given, but now in negative x-direction for 5 s. Afterwards, an angular velocity of 1 m/s is published for 5 s, first in positive z-direction and then in negative z-direction, which is what causes the horizontal panning of the sub-image. This loop continues until the user stops the node.

```
<launch>
    <node pkg="cam2image_vm2ros" exec="cam2image" name="cam2image" args="--ros-args --params-file src/cam2image_vm2ros/config/cam2image.yaml"/>
    <node pkg="relbot_simulator" exec="relbot_simulator" name="relbot_simulator" args="--ros-args -p use_twist_cmd:=true"/>
    <node pkg="sequence_generator_pkg" exec="sequence_generator_node" name="sequence_generator_node"/>
</launch>
```

Figure 11: Launch file for the cam2image, relbot simulator and sequence generator nodes.

Figure 11 shows the launch file for the required packages for this code. It can also be seen that the ros arguments needed to properly run the code are included. The launch file that sets up the `cam2image`, `relbot_simulator` and `sequence_generator_node` nodes can be ran using the following command:

```
ros2 launch sequence_generator_pkg sequence_generator_launch.xml
```

The rqt graph of this system is shown in Figure 12.

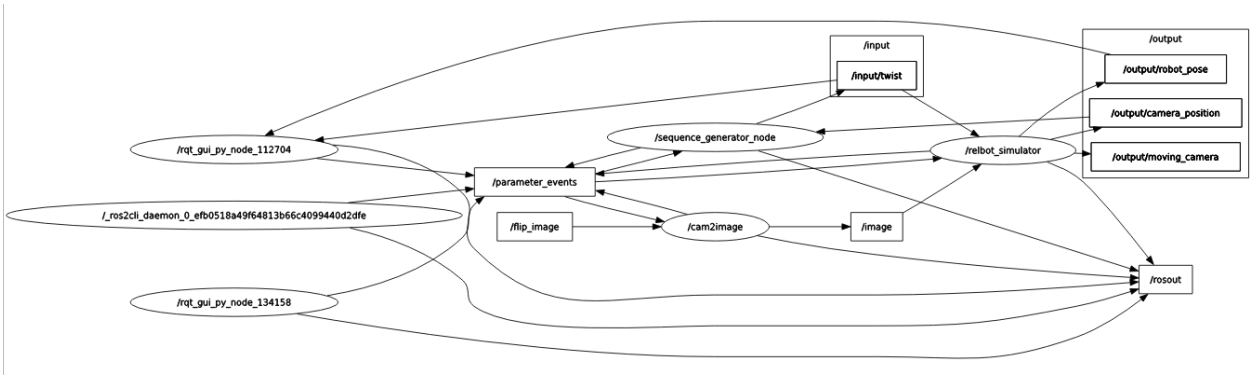

Figure 12: Rqt graph of the cam2image, RELbot simulator, and sequence generator nodes running. Also, the rqt plot gui is visible, which plots the input twists and output robot poses.

Figure 12 showcases that the `sequence_generator_node` publishes the `input/twist` to both the `RELbot_simulator_node` and the rqt plot gui. This gui is also subscribed to the `output/robot/pose` topic, resulting in the graph seen in Figure 13.

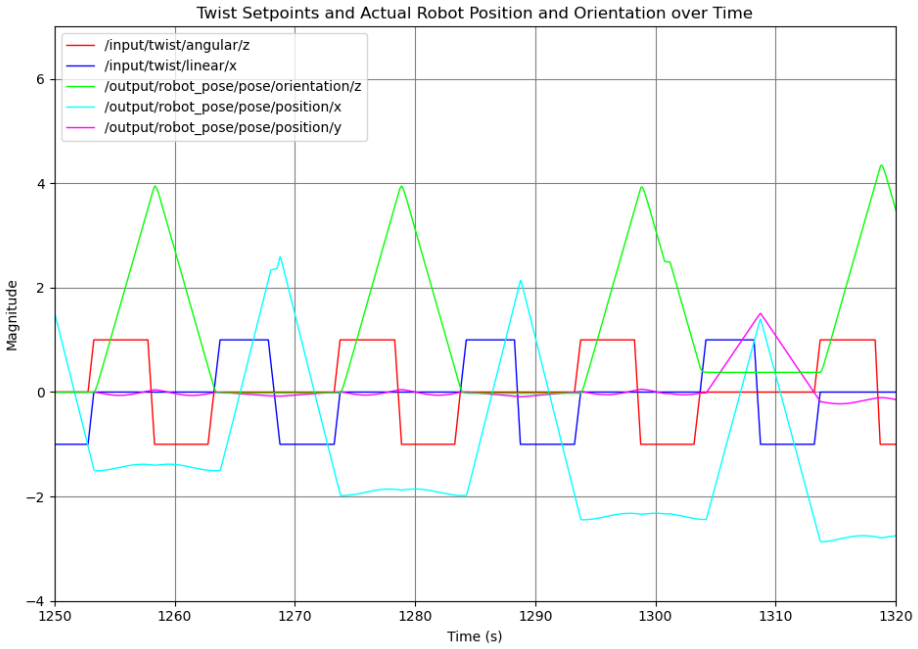

Figure 13: Twist Setpoints and Actual Robot Position and Orientation over Time.

Figure 13 shows the twist setpoints that are used as inputs for the `relbot_simulator` and the outputs of this node, the robot position and orientation. It becomes clear that a positive linear twist in the x-direction causes a linear increase in the x-position of the robot. Which is as expected, because a zoom corresponds to a translation of the RELbot in x-position. Besides, it shows that a positive angular twist in z-direction causes a linear increase in the z-orientation of the robot. Lastly, there are some anomalies in the results, there are some small disturbances in the z-orientation and x-position of the robot and a random peak in y-position of the robot. It can also be seen that the x-position slowly drifts negatively in magnitude.

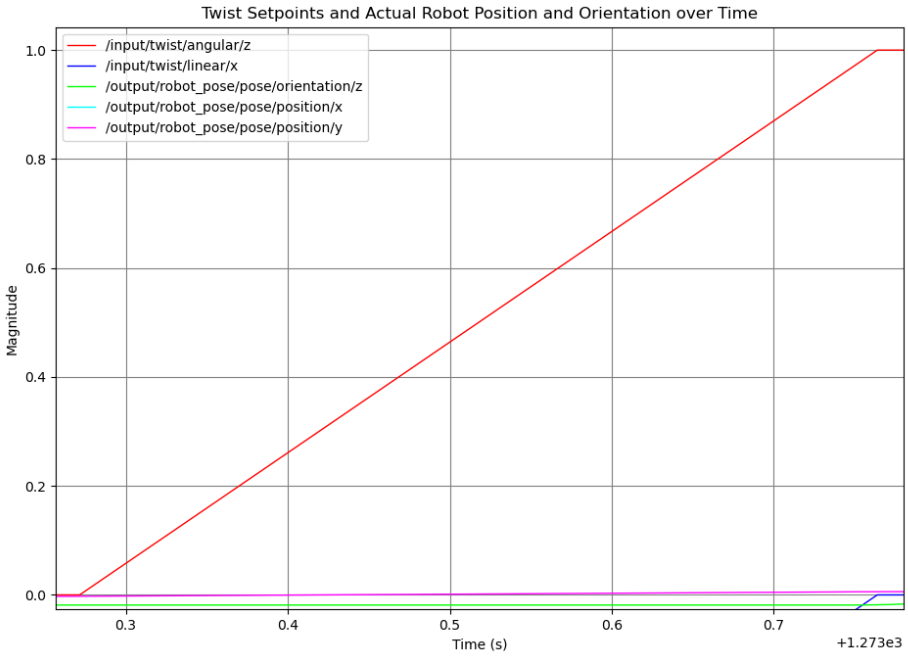

Figure 14: Showcase of response of the system to a step input.

The red graph in Figure 14 showcases the response of the system to a step input. It starts at $t \approx 2.75\,\mathrm{s}$, and at $t \approx 5.75\,\mathrm{s}$ it has reached $1 - e^{-1} \approx 0.63$ of the final value. This time difference of $5.75 - 2.75 = 3\,\mathrm{s}$ is equal to the time constant of the system.

Integration of image processing and RELbot Simulator

To extend the `sequence_generator` node such that it is able to follow the center of gravity of the brigh light detected by the `object_detection` node, a subscription is made to the `CoG` and `output/camera_position` topics. The pixel coordinates of the bright light CoG and subimage camera position are continuously updated and the difference is taken. Figure 15 shows an additional line in the launch file to also run the `object_detection` node. It also still possible to run the old functionality by setting the parameter `assignment_selector` to false in the parse parameter function and then building the package again.

```
<launch>
    <node pkg="cam2image_vm2ros" exec="cam2image" name="cam2image" args="--ros-args --params-file src/cam2image_vm2ros/config/cam2image.yaml"/>
    <node pkg="relbot_simulator" exec="relbot_simulator" name="relbot_simulator" args="--ros-args -p use_twist_cmd:=true"/>
    <node pkg="object_detection_pkg" exec="object_detection_node" name="object_detection_node"/>
    <node pkg="sequence_generator_pkg" exec="sequence_generator_node" name="sequence_generator_node"/>
</launch>
```

Figure 15: Launch file for the cam2image, relbot simulator, object detection and sequence generator nodes.

The sequence generation of the horizontal panning is done using the following twist:

```
random_twist.angular.z = scaling_factor_theta_*pix_diff_x/dt_;
```

The sequence generation is different when zooming, because it must zoom in when it thinks it is outside of the image. It must zoom out when the CoG is between 60 and 180 in absolute y-pixel coordinates such that the bright light is inside of the subimage. Besides, it must zoom in when the bright light is between 10 and 60 in y-pixel coordinates. Lastly, when it lower than ten it is chosen to zoom out, because it would otherwise

zoom too much. This is implemented as follows, where the default value of the scaling value in x(zoom) is 0.3:

```
if ((pix_diff_y < 180 && pix_diff_y > 60) || (pix_diff_y < 10))
{
    random_twist.linear.x = -scaling_factor_x_/dt_;
} else
{
    random_twist.linear.x = scaling_factor_x_/dt_;
}
```

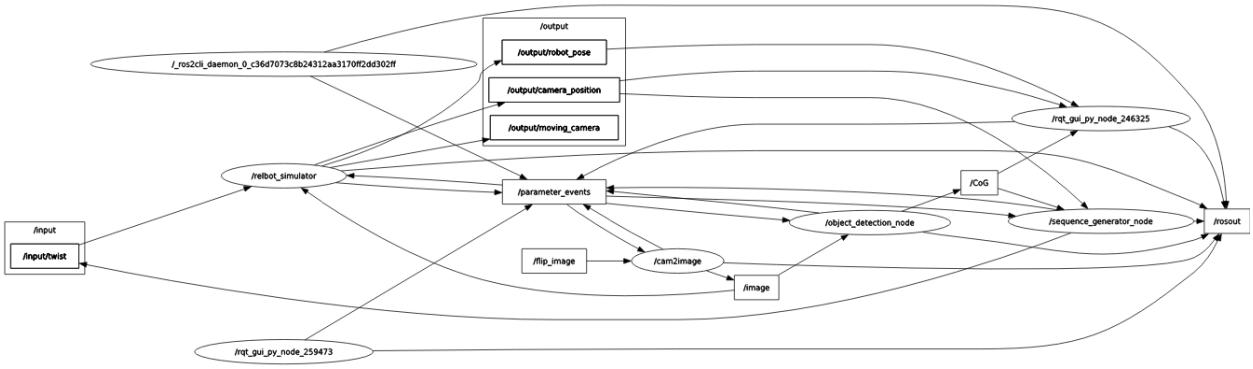

Figure 16: Rqt graph of the cam2image, RELbot simulator, object detection and sequence generator nodes.

Figure 16 shows the rqt graph when running the cam2image, RELbot simulator, object detection and sequence generator nodes. The `sequence generator` node is subscribed to the `camera_position` and `CoG` topics. The `object_detection` and `relbot_simulator` packages are subscribed to the `image` topic. Whereas the `relbot_simulator` is also subscribed to the `input/twist` topic.

Figure 17: Pixel coordinates of the center of gravity of the bright light and subimage over time.

Figure 17 shows the x-coordinate of the center of the subimage is able accurately track the x-coordinate of the center of the bright light, however only with a delay of approximately 1 seconds. It can also be seen that the camera position in y-direction cannot be influenced.

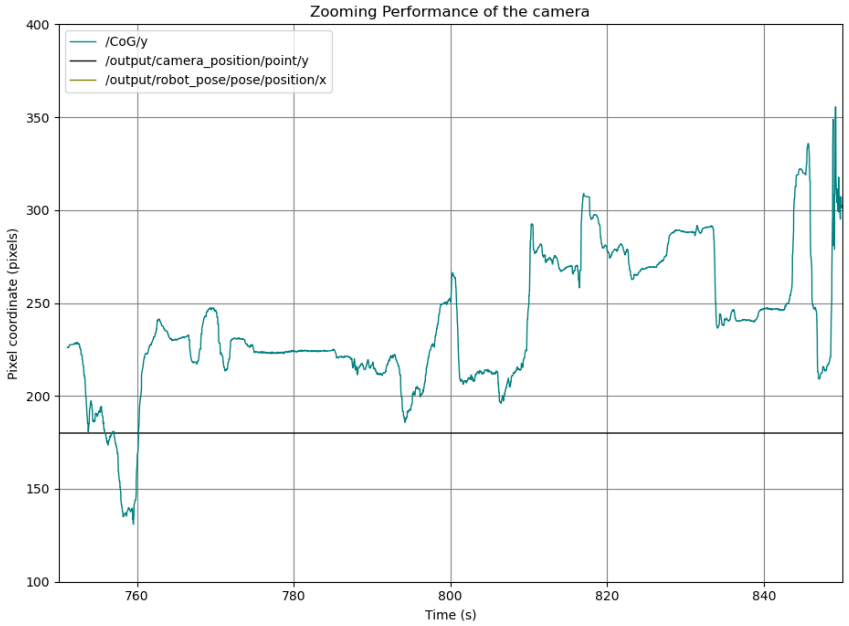

Figure 18: Pixel y-coordinate of the center of gravity of the bright light over time.

Figure 18 shows the center of gravity of the bright light over time. It can be seen that it also reaches out

of the working range of 240 pixels in y-direction.

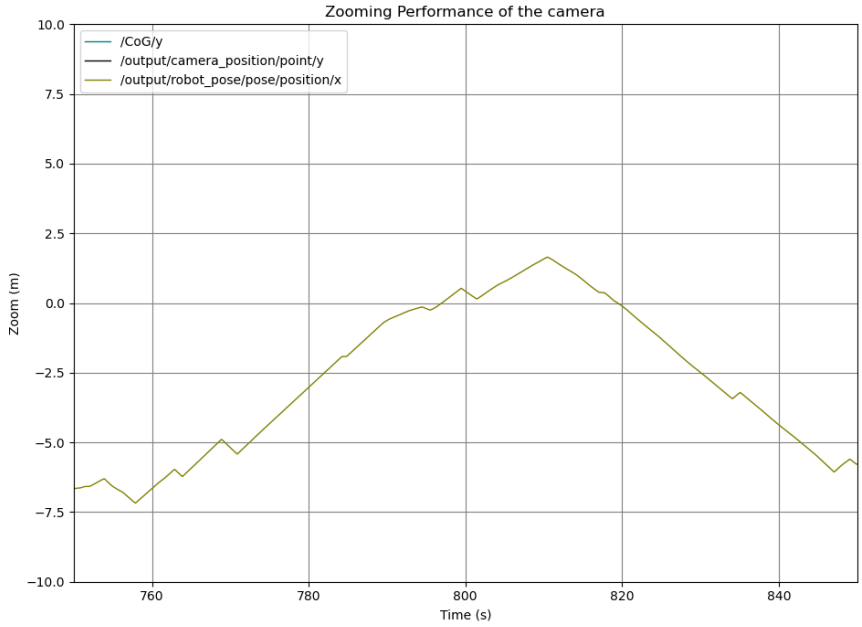

Figure 19: Zoom parameter x over time.

Figure 19 shows the evolution of the zoom parameter x over time. It can be seen that it is able to reach out of the actual working range of the RELbot simulator. After all, the goal of zooming out when the light is out of the sub-image until it is in the sub image, was not fully reached. It is not clear from these functions, however it does account for differences in y-coordinate of the pixels, but in a very slow manner.

The sequence controller is supposed to generate sequences of velocity set-points for continuously updating desired end points. Normally, this is done using an open-loop control system with time-driven events. However, in this implementation, an event-driven approach is taken to follow the desired point. What is meant by this, is that the coordinates of the camera sub-image are fed back into the sequence controller to generate constant set-points with durations of 1 second. This is a feedback loop and therefore this implementation is a closed loop system.

Sequence controllers can be soft realtime and hard realtime. Therefore, when the application of the sequence controller is a soft realtime application, ROS2 is a good option to implement such a controller. However, in hard realtime applications, ROS2 could possible not be fast enough in handling messages and could miss timings. This could lead to ROS2 missing important messages and causing time delays, which in their turn could possibly cause failure of the system.

References

[1] ROS Wiki. *Converting between ROS images and OpenCV images (C++)*. 2017. URL: `http://wiki.ros.org/cv_bridge/Tutorials/UsingCvBridgeToConvertBetweenROSImagesAndOpenCVImages`.

[2] OpenCV. *Color Space Conversions*. 2025. URL: `https://docs.opencv.org/3.4/d8/d01/group__imgproc__color__conversions.html`.

[3] OpenCV. *Image Thresholding*. 2025. URL: https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html.

[4] OpenCV. *cv::Moments Class Reference*. 2025. URL: https://docs.opencv.org/3.4/d8/d23/classcv_1_1Moments.html.

[5] wikipedia. *Image moment*. 2024. URL: https://en.wikipedia.org/wiki/Image_moment.

[6] OpenCV. *High-level GUI*. 2025. URL: https://docs.opencv.org/4.x/d7/dfc/group__highgui.html#ga453d42fe4cb60e5723281a89973ee563.